

When Compilers Meet Agents: A Multi-Stage Architecture for AI-Driven C-to-Rust Migration

By Dmitry Rudakov

Organizations managing millions of lines of legacy C code face a significant technical challenge: evolving expertise in systems programming, combined with regulatory mandates for memory-safe languages and the emerging need for automated migration solutions.

This article introduces a multi-agent architecture for automated C-to-Rust migration that draws on compiler science and graph algorithms, and is designed to produce verifiable, Idiomatic Rust at repository scale. The architecture combines academic foundations from nanopass compilers to graph community detection with the agentic capabilities of today's frontier AI models to tackle a problem that even the largest technology companies have not yet solved.

The Scale of the Problem: Declining Expertise and Rising Demand

Three forces are converging into a significant software engineering challenge—and opportunity.

Systems programming expertise is shifting. The population of expert C systems programmers is contracting. Senior C developers are transitioning to other roles; fewer graduates pursue systems-level programming. Rust enterprise adoption has grown substantially in recent years, while C's market share has shifted. The skill landscape has changed: it is now easier to hire a Rust developer than a C specialist.

Regulatory landscape is evolving. In February 2024, the *White House Office of the National Cyber Director* published a report formally recommending memory-safe languages for critical infrastructure. By December 2024, *CISA* and the *FBI* issued a memory-safety mandate with enforcement dates for critical systems. Best-practice recommendations are becoming compliance requirements.

AI models have reached a new level of capability. Today's frontier models—*Claude Opus 4.6*, *GPT-5.3*, *Gemini 3 Pro*—are a generation beyond what was available even a year ago. They execute tools reliably, maintain coherence across thousands of lines of context, self-correct without human intervention, and operate in plan-execute-verify-correct loops. These capabilities make AI a viable collaborator on complex, multi-step engineering tasks.

Yet despite billions in investment, dozens of academic papers, and a US government research program dedicated to this exact problem, no robust, production-ready automated C-to-Rust migration solution exists today.



The Industry Landscape: No Solution Yet

Microsoft: Ambition Meets Reality

In late 2025, a [LinkedIn hiring post](#) from a *Microsoft* Distinguished Engineer attracted wide attention with an ambitious goal to eliminate *C* and *C++* from *Microsoft* by 2030. Within days, the post was [updated with a clarification](#): it is a research project building migration tooling, not a plan to rewrite *Windows* with *AI*.

Microsoft's approach - combining algorithmic program analysis with *AI* agents for translation and repair - is directionally similar to the architecture described in this article. But at the time of writing, it remains a research charter, not a shipped capability.

DARPA: A Formal Research Priority

DARPA launched the [TRACTOR program](#) (*Translating All C to Rust*), explicitly acknowledging that reliable automated *C*-to-*Rust* translation is an unsolved research problem. As [DARPA's own announcement](#) put it: the challenge is to «dramatically improve the automated translation from *C* to *Rust*, particularly for program constructs with the most relevance.»

The program's [first contract went to a multi-university consortium](#) building formally verified lifting tools - confirming that even the *US* defense establishment considers this a multi-year effort.

The Academic Wave of 2024–2025

The past two years saw a significant surge in *C*-to-*Rust* research. Over a dozen significant papers appeared across top venues, each attacking a different slice of the problem:

- **Multi-agent pipelines:** *Rustify* (ICLR 2026), *ACToR* (Li et al., 2025), and *AlphaTrans* (FSE 2025) showed that specialized agents collaborating in sequence outperform monolithic *LLM* approaches.
- **Neuro-symbolic hybrids:** *C2SaferRust* (Columbia, 2025) and *SafeTrans* (2025) combined static analysis with *LLM* repair loops.
- **Skeleton-guided translation:** *EvoC2Rust* (Wang et al., accepted at ICSE 2026) decomposed projects into compilable skeletons before filling in function bodies.
- **Dual code-test translation:** *Szygy* (Shetty et al., 2024) demonstrated translating tests alongside code for equivalence checking.
- **Dependency-guided ordering:** *RustMap* (Cai et al., 2025) used program analysis to determine translation sequence.
- **Comprehensive benchmarks:** *CRUST-Bench* (Khatry et al., COLM 2025) revealed the difficulty of repository-level *C*-to-*Rust* tasks in single-shot settings.

Despite this progress, a survey of the landscape revealed that no existing approach addresses several key technical requirements simultaneously: workload partitioning for *LLM* context limits,



structural drift detection, ordered dimension-separated transformation, or multi-graph analysis for research versus translation.

The architecture in this article addresses that gap.

The Architecture: Five Pillars for Reliable Migration

The Core Insight: C and Rust Have Different Structural Models

Before diving into the architecture, it helps to understand why direct translation fails.

C code has three loosely coupled concerns:

- **Macros:** compile-time code generation
- **Types:** structs, enums, typedefs
- **Code:** functions operating on types

These concerns exist in separate scopes—a `#define` has nothing to do with a `struct`, which has little coupling to the function that uses it.

Rust tightly couples all three: types have methods (`impl` blocks), behavior is shared through traits, and macros are procedural, not textual.

When an *LLM* is asked to translate a *C* file in one pass, it must simultaneously resolve macro expansions, type mappings, and code logic. The result is often type fragmentation, compilation failures, and «*C* written in *Rust* syntax.»

The architecture solves this by processing each concern in strict order:

Macros first → Types second → Code third

This formula captures the central design decision: **types must be unified before code translation begins**. It is a direct application of the *nanopass compiler pattern*, where complex transformations are decomposed into many small, single-purpose passes. Without ordered dimension separation, the *LLM* translating each function independently produces fragmented type definitions; with it, a unified type system emerges across the entire codebase.

Pillar 1: Layered and Ordered Translation

Each concern gets its own analysis pass and its own graph:

- **Macros:** Handles `#define`, X-macros, *BSD* queues. Output: Explicit code (no macros in *Rust*).
- **Types:** Handles structs, enums, typedefs. Output: Unified *Rust* types with all fields.
- **Code:** Handles functions, control flow. Output: Functions using unified types.



The order is not optional. Translating code before types causes each function to invent its own local type definitions. Translating types before expanding macros leaves undefined types from macro expansions. The architecture enforces the sequence:

Macros → Types → Code.

Pillar 2: Intelligent Batching via Community Detection

A large codebase cannot be translated in one *LLM* session. The partitioning strategy is essential to effective translation.

File boundaries are arbitrary—they don't reflect architectural structure. Dependency-ordered traversals create long sequential chains. Random chunking breaks semantic relationships.

The architecture uses **Leiden community detection** — an algorithm from network science—that finds tightly-connected clusters in graphs. Applied to the call graph of a *C* codebase, it identifies natural «communities» of functions that call each other heavily but have few connections to the outside.

Each community becomes a translation batch of manageable size — large enough for meaningful context, small enough for reliable *LLM* processing. Communities can be translated in parallel if they are independent.

The *Leiden* algorithm provides a key mathematical guarantee: all communities it produces are connected. This means no community ever groups functions that have no call relationship — a failure mode that would produce broken translation units.

Pillar 3: Structural Verification

After translating a community, integrity verification is essential to ensure correctness and completeness.

The architecture compares the call graph of the original *C* code with the call graph of the generated *Rust* code using **Jaccard similarity** — a standard set-overlap metric:

$$J(C_calls, Rust_calls) = |intersection| / |union|$$

Score $\geq$ threshold → Proceed

Score $<$ threshold → Structural drift detected → trigger recovery

This catches a class of errors that compilation alone misses: the code compiles, tests pass, but the *LLM* quietly restructured the call hierarchy, merged functions, or dropped internal calls. Left undetected, structural drift can undermine an otherwise correct migration.

Pillar 4: Source Traceability

Every translated *Rust* function carries a breadcrumb comment linking it back to its *C* origin:



```
/// Translated from: src/protocol/session.c:466-496 (start_transmit_timer)
/// C prototype: void start_transmit_timer(struct session_ctx *bs)
pub unsafe fn start_transmit_timer(bs: *mut SessionCtx) {
    // Implementation...
}
```

This serves three purposes:

- Later-stage agents can consult the original C for ambiguous semantics.
- Human reviewers can do side-by-side verification.
- The traceability itself can be automatically verified (100% coverage is a goal).

Pillar 5: Tiered AI Processing

Error fixing needs two fundamentally different skills: diagnosis (what's wrong and how to fix it) and execution (apply the fix mechanically).

Using a frontier reasoning model for both tasks is unnecessarily expensive. The architecture pairs model tiers:

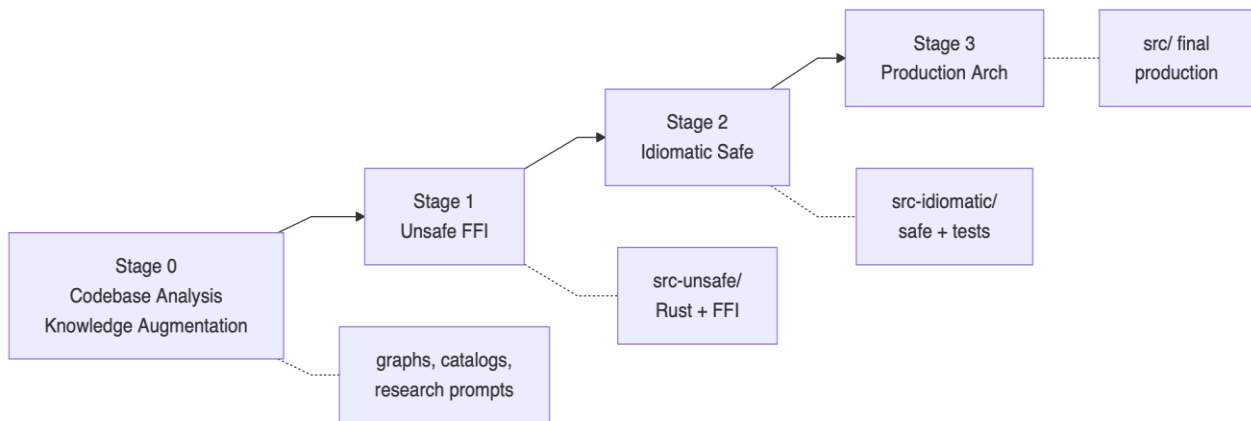
Error detected

- Reasoning Model (e.g., Claude Opus 4.6):
 1. Classify error type
 2. Triage priority (blocking vs. deferrable)
 3. Generate fix pattern with application order
- Fast Model (e.g., Claude Sonnet 4):
 - Apply fix pattern exactly as specified
- Verify (compile + test)
- Success? Next batch : Escalate back to Reasoning

The reasoning model's output specifies not just «what's wrong» but «how to fix it, in what order, and what to watch for.» This structured guidance lets the fast model execute reliably without needing to reason about strategy.



The Pipeline: Four Stages



Stage 0 builds foundational knowledge by scanning the codebase, discovering patterns (queues, macros, framework idioms, protocols, etc.), building call graphs and type dependency graphs, and running automated literature research to find relevant migration patterns.

Stages 1–3 are where the migration happens — and their separation is the architecture's key design decision.

Why Three Stages? The Staged Translation Rationale

Direct translation from *C* to *Idiomatic Rust* in one step presents significant architectural challenges.

Part 1: Unsafe Rust enables gradual, production-testable migration. Stage 1 produces *FFI*-compatible unsafe *Rust* — code that can be compiled and linked directly against the remaining *C* code. This means one community can be migrated at a time, a mixed *C* + *Rust* binary can be built after each batch, and the existing test suite can be run against it.

Community 1 (types) → Compile → FFI test with *C* → ✓ Pass

Community 2 (core) → Compile → FFI test with *C* → ✓ Pass

Community 3 (io) → Compile → FFI test with *C* → ✗ Fail → Fix while context fresh

Errors are caught immediately, while the relevant code is still in the model's context window. This is significantly more efficient than discovering integration failures after translating the entire codebase.

This staged *FFI* approach uses *SACTOR* (Zhou et al., 2025) as a foundation, extending it with community-level batching, dimension ordering, and structural verification between stages.

Part 2: Translating Unsafe Rust → Idiomatic Rust is far easier than C → Idiomatic Rust. When Stage 2 begins, the *LLM* receives three aligned artifacts for every function:

- **Original C code:** Ground truth for semantics and algorithm intent.
- **Unsafe Rust:** Already valid *Rust* syntax—no language barrier to cross.



- **Traceability link:** Exact line mapping between *C* and *Rust* for cross-reference.

Transforming unsafe *Rust* to *Idiomatic Rust* is a within-language refactoring task: the syntax is already *Rust*, the compiler provides type inference that *C* lacks, and changes can be applied one pattern at a time (raw pointers → references, return codes → `Result<T, E>`, globals → `OnceLock`).

Meanwhile, the original *C* source is still available as a second verification channel. If the *LLM* introduces a subtle bug during idiomatization, the traceability links let verification agents compare the *Idiomatic Rust* against the original *C* algorithm—catching issues that would be invisible if only the *Rust* version existed.

This double-stage verification provides complementary coverage:

- Stage 1 catches integration and runtime errors via *FFI* testing.
- Stage 2 catches semantic discrepancies and logic bugs via traceability-enabled comparison with the original *C*.

C2SaferRust (Nitin et al., 2025) follows a related pattern—generating unsafe *Rust* via *C2Rust*, then refining slices with an *LLM*. The architecture described here builds on both *SACTOR* and *C2SaferRust*, adding community-level *FFI* testing, three-artifact alignment for the idiomatization stage, and *Jaccard*-based structural verification between stages.

Part 3: Stage 3 handles deep architectural redesign—a different class of problem. Stage 3 is optional but addresses transformations that are qualitatively different from line-by-line translation. These include:

- Replacing *C*'s `select()/poll()` patterns with *Rust*'s `async/await` and `tokio` runtime.
- Converting *C* pthread multi-threading to *Rust* `async` tasks or `rayon` thread pools.
- Extracting trait hierarchies from *C* function-pointer tables.
- Reorganizing flat *C* file layouts into *Idiomatic Rust* module trees.

These are architectural changes, not translation changes. They require a *Premium*-tier reasoning model that can reason about system design, concurrency patterns, and the performance implications of different *Rust* abstractions. Most migration projects complete at Stage 2 with clean, safe *Rust* code; Stage 3 is for organizations that want production-grade, *Idiomatic Modern Rust* that leverages the language's full capabilities.

From Architecture to Code: A PydanticAI Implementation

The architecture is framework-agnostic by design. Reference implementations utilize [PydanticAI](#) — a type-safe *Python* framework for building *AI* agents—together with **pydantic-graph** for state-machine orchestration and **NetworkX** for graph analysis.

The following examples illustrate how the architecture's most distinctive concepts—community detection, dimension-ordered translation, and structural verification—translate into concrete *Python* code.



Community Detection: Partitioning the Call Graph

The type graph builder parses *C* headers first (for complete dependency information), then *Rust* sources (for file locations), and uses *Louvain* community detection to partition types into translation batches:

```
import networkx as nx
from networkx.algorithms import community as nx_community

class TypeGraphBuilder:
    """
    Strategy:
    1. Parse C headers FIRST – these have complete dependency info
    2. Parse Rust files – get locations for matching type names
    3. Merge by name – C provides deps, Rust provides locations
    4. Build NetworkX graph for community detection
    """

    def build_networkx_graph(self) -> nx.DiGraph:
        G = nx.DiGraph()
        for name, info in self.types.items():
            G.add_node(name, **info.to_dict())
            for dep in info.dependencies:
                if dep in self.types and dep != name:
                    G.add_edge(name, dep)
        return G

    def compute_communities(self) -> dict[str, int]:
        """Partition types using Louvain community detection."""
        G = self.build_networkx_graph()
        G_undirected = G.to_undirected()

        communities = nx_community.louvain_communities(G_undirected, seed=42)
        return {
            node: i
            for i, comm in enumerate(communities)
            for node in comm
        }

    def compute_levels(self) -> dict[int, list[str]]:
        """Topological sort on SCCs → dependency levels for translation order."""
        G = self.build_networkx_graph()
        sccs = list(nx.strongly_connected_components(G))
        # Contract SCCs into super-nodes, topological sort the DAG
```




```
# Types with no dependencies → Level 0; dependents → Level N+1
...
```

Dimension-Ordered Stage 1: Macro → Type → Code Phases

The pipeline enforces the functor composition order using *pydantic-graph* nodes. Each phase dispatches communities sequentially, and the next phase only starts when the previous one completes:

```
from pydantic_graph import BaseNode, GraphRunContext, End

class Stage1StartNode(BaseNode[MigrationState, MigrationDeps, MigrationResult]):
    """Entry point for Stage 1: Unsafe FFI Translation."""

    async def run(self, ctx: GraphRunContext) -> Stage1MacroPhaseNode | End:
        stage0 = ctx.state.get_stage_progress(0)
        if stage0.status != StageStatus.COMPLETE:
            return self.create_failure_result(ctx, "Stage 0 must complete first")

        logger.info(
            f"Stage 1 communities: "
            f"macro={len(ctx.state.macro_communities)}, "
            f"type={len(ctx.state.type_communities)}, "
            f"code={len(ctx.state.code_communities)}"
        )
        return Stage1MacroPhaseNode() # Always start with macros

class Stage1MacroPhaseNode(BaseNode[MigrationState, MigrationDeps,
MigrationResult]):
    """MACRO phase – processes all macro communities, then hands off to TYPE
    phase."""

    async def run(self, ctx: GraphRunContext) -> MacroTranslatorNode |
Stage1TypePhaseNode:
        pending =
ctx.state.get_pending_communities_for_category(GraphCategory.MACRO)
        if not pending:
            return Stage1TypePhaseNode() # → next dimension
        return MacroTranslatorNode(community_id=pending[0])

# After macros complete → types; after types complete → code
# The graph structure enforces: F_macros → F_types → F_code
```



Jaccard Verification: Structural Quality Gate

After all three phases complete, the pipeline compares the *C* and *Rust* call graphs and gates progress on structural preservation:

```
class Stage1JaccardNode(BaseNode[MigrationState, MigrationDeps, MigrationResult]):  
    """  
    Jaccard similarity between C Code Graph and Unsafe Rust Graph.  
    Threshold: configurable. Retries on failure.  
    """  
    retry_count: int = 0  
  
    async def run(self, ctx: GraphRunContext) -> Stage1CompleteNode |  
    Stage1JaccardRetryNode | End:  
        c_graph = ctx.state.code_graph  
        rust_graph = ctx.state.get_stage_progress(1).metrics["unsafe_rust_graph"]  
  
        #  $J(C, Rust) = |intersection| / |union|$   
        jaccard_score, diff = await self._calculate_jaccard(c_graph, rust_graph)  
  
        if jaccard_score >= JACCARD_THRESHOLD:  
            return Stage1CompleteNode(jaccard_score=jaccard_score)  
  
        if self.retry_count < MAX_RETRIES:  
            return Stage1JaccardRetryNode(  
                retry_count=self.retry_count + 1,  
                diff_info=diff,          # Feeds back to triage agent  
            )  
  
        return End(MigrationResult(  
            success=False,  
            summary=f"Structural drift detected",  
        ))
```

These three examples show the architecture's distinctive mechanisms — community partitioning, dimension ordering, and structural gating — expressed as typed *Python* using *PydanticAI* agents and *pydantic-graph* state machines.

Established Approaches and Introduced Innovations

This section outlines the academic origins of the architecture's components and identifies its novel contributions.



Directly Adopted from Prior Research

The following components draw from established literature:

- **The nanopass compiler pattern:** Complex transformations work best as many small, single-purpose passes.
- **Staged unsafe-to-idiomatic pipelines:** *SACTOR* (Zhou et al., 2025) uses a two-stage approach of producing *FFI*-compatible unsafe *Rust* first, then refining it to idiomatic code. *C2SaferRust* follows a similar path using *C2Rust* as the first stage.
- **Compiler-in-the-loop repair:** Feeding compilation errors back to the *LLM* for iterative fixing appears in *SafeTrans*, *EvoC2Rust*, *Rustine*, and others.
- **Leiden and Louvain community detection:** Well-established in graph theory and network science.
- **Jaccard similarity:** A standard metric used in clone detection and information retrieval.

Introduced by This Architecture

The following represent novel contributions:

- **Dimension separation as an ordered pipeline:** While individual papers handle macros, dependencies, or types, no surveyed work treats *C*-to-*Rust* migration as three explicitly ordered transformation dimensions that must execute in a fixed sequence.
- **Community detection for LLM workload partitioning:** Existing approaches use file-by-file processing, dependency-graph *DFS*, reverse call-graph order, or semantic chunking. None apply graph community detection algorithms for partitioning.
- **Jaccard similarity as a migration quality gate:** Comparing call graphs between *C* source and *Rust* output to detect structural drift in *LLM*-generated translations is a novel application.
- **Trait-based dependency injection for C macros:** The mapping of *C*'s redefinable `#define` patterns to *Rust* traits with swappable implementations appears to be novel.
- **Unified-to-separated graph architecture:** One graph for the research phase (all symbols together), then three dimension-specific graphs for translation (macros, types, code), and separate comparison graphs for verification.

Practical Implementation

The *C*-to-*Rust* migration market is beginning to mature. *Microsoft*, *DARPA*, and significant academic effort signal convergence on multi-stage, AI-assisted approaches. Currently, no production-ready commercial solution exists; available tools are either research prototypes or produce non-idiomatic code requiring extensive manual refinement.

The complete pipeline is orchestrated as a **pydantic-graph** state machine where each node is a typed step that returns the next node or terminates. This design pattern captures all three translation stages within a single declarative structure:



```
from pydantic_graph import Graph
```

```
migration_graph: Graph[MigrationState, None, MigrationResult] = Graph(
    nodes=[
        Stage0Node,
        # Stage 1: Three-phase (MACRO → TYPE → CODE)
        Stage1StartNode,
        Stage1MacroPhaseNode, MacroTranslatorNode, Stage1MacroPhaseCompleteNode,
        Stage1TypePhaseNode, TypeTranslatorNode, Stage1TypePhaseCompleteNode,
        Stage1CodePhaseNode, CodeTranslatorNode, Stage1CodePhaseCompleteNode,
        Stage1BuildGraphNode, Stage1JaccardNode, Stage1JaccardRetryNode,
        Stage1CompleteNode, StagnationHandlerNode,
        # Stage 2: Three-phase idiomatization (MACRO → TYPE → CODE)
        Stage2Node,
        Stage2MacroPhaseNode, MacroIdiomatizerNode, Stage2MacroPhaseCompleteNode,
        Stage2TypePhaseNode, TypeIdiomatizerNode, Stage2TypePhaseCompleteNode,
        Stage2CodePhaseNode, CodeIdiomatizerNode, Stage2CodePhaseCompleteNode,
        Stage2BuildGraphNode, Stage2JaccardNode, Stage2JaccardRetryNode,
        Stage2CompleteNode, Stage2StagnationHandlerNode,
        # Stage 3: Conceptual architecture
        Stage3Node,
        Stage3AsyncPhaseNode, AsyncTransformNode,
        Stage3AsyncPhaseCompleteNode,
        Stage3TraitPhaseNode, TraitExtractorNode,
        Stage3TraitPhaseCompleteNode,
        Stage3ModulePhaseNode, ModuleDecomposerNode,
        Stage3ModulePhaseCompleteNode,
        Stage3TypestatePhaseNode, TypestateImplementerNode,
        Stage3TypestatePhaseCompleteNode,
        Stage3DocPhaseNode, DocGeneratorNode,
        Stage3DocPhaseCompleteNode,
        Stage3CompleteNode, Stage3StagnationHandlerNode,
    ],
)
```

Looking Ahead

The field is advancing rapidly as model capabilities mature and context windows expand.

Key developments to watch:

- **Pointer Knowledge Graphs:** Richer pointer semantics for complex data structures
- **Bridged debuggers:** Runtime equivalence checking across language boundaries



- **Experience bases:** Accumulated repair patterns across migrations
- **Framework orchestration:** *LangGraph*, *Microsoft Agent Framework* for production deployments
- **Formal verification integration:** *DARPA's ForCLift* for safety-critical subsystems

The question is no longer whether *AI* will automate *C-to-Rust* migration. It is which architecture will do it reliably, at scale, and at reasonable cost.

The architecture described here—dimension-separated, community-partitioned, structurally verified, and tiered across model capabilities—presents a practical solution, built on established academic foundations and extended with targeted innovations where the literature had gaps.

References

Foundational:

- Sarkar, Waddell & Dybvig (2004). [A Nanopass Framework for Compiler Education](#). *JFP*.
- Wadler (1998). [The Expression Problem](#).
- Blondel et al. (2008). [Fast unfolding of communities in large networks](#). *arXiv:0803.0476*.
- Traag, Waltman & van Eck (2019). [From Louvain to Leiden](#). *Scientific Reports* 9, 5233.

C-to-Rust Migration:

- Hong & Ryu (2025). [Automatically Translating C to Rust](#). *ACM CACM* 68(11).
- Zhang et al. (2023). [Ownership Guided C to Rust Translation](#). *CAV* 2023.
- Emre et al. (2021). [Translating C to Safer Rust](#). *OOPSLA* 2021.
- Emre et al. (2023). [Aliasing Limits on Translating C to Safe Rust](#). *OOPSLA* 2023.
- Nitin et al. (2025). [C2SaferRust](#). *arXiv:2501.14257*.
- Shetty et al. (2024). [Syzygy](#). *arXiv:2412.14234*.
- Wang et al. (2025). [EvoC2Rust](#). *ICSE 2026 SEIP*.
- Cai et al. (2025). [RustMap](#). *arXiv:2503.17741*.
- Zhou et al. (2025). [SACTOR](#). *arXiv:2503.12511*.
- Dehghan et al. (2025). [Rustine](#). *arXiv:2511.20617*.
- SafeTrans (2025). [arXiv:2505.10708](#).

Related Work:

- Khatri et al. (2025). [CRUST-Bench](#). *COLM* 2025.

Government & Industry:

- White House *ONCD* (2024). [Back to Building Blocks](#).
- *CISA/FBI* (2024). Memory-safe language mandate.
- *DARPA TRACTOR*. [darpa.mil/program/translating-all-c-to-rust](#).
- *Microsoft* (2025). [Rust migration hiring and clarification](#).



Dmitry Rudakov

Solution Architect and Innovation Leader at Capgemini Engineering Eastern Europe GEC.

He has 20+ years of experience in the IT industry, with focus on architectural innovation across enterprise platforms, distributed systems, and emerging technologies. His expertise spans nearly every industry vertical and technology stack—from distributed systems and telecommunications to cloud-native architectures and AI/ML platforms.

Over the last two years, Dmitry has focused on prompt-first generative AI development—exploring how large language models can be orchestrated as autonomous engineering agents rather than passive assistants. His work on the C-to-Rust multi-agent migration architecture combines compiler science and graph algorithms with practical agentic AI patterns, demonstrating how structured, multi-stage pipelines can address the complexity of large-scale code migration.